

Robust Hierarchical Moving Horizon Planner

The moving horizon planner provides a reference path and selects the operating region the controller should operate in. To calculate a safe passage—satisfy the constraints and avoid all obstacles—the planner takes the uncertainty bounds corresponding to the different operating modes directly into account.

Definitions

Define State-space Model

A double integrator model is used as the dynamics of the UAV. The equations are

$$\begin{aligned} x_p(k+1) &= A_p x_p(k) + B_p u_p(k) \\ y_p(k) &= C x_p(k) \end{aligned}, \text{ where}$$

$$x_p = \begin{bmatrix} x \\ \dot{x} \\ z \\ \dot{z} \end{bmatrix}$$

$$u_p = \begin{bmatrix} \ddot{x} \\ \ddot{z} \end{bmatrix}$$

$$y_p = \begin{bmatrix} x \\ z \end{bmatrix}.$$

```
clear
M=3;
dt = 0.01;

A_c = [1 dt 0 0;
        0 1 0 0;
        0 0 1 dt;
        0 0 0 1];

A_p = A_c^M;

B_c = [dt^2/2 0;
        dt 0;
        0 dt^2/2;
        0 dt];

B_p = zeros(size(B_c));
for i = 0:M-1
    B_p = B_p + A_c^i*B_c;
end

C_p = [1 0 0 0;
        0 0 1 0];
```

Define Square Obstacle Parameters

$$O_l = \{y|E_l y \leq f_l\}$$

```
E_l = [1 0;
        -1 0;
        0 1;
        0 -1];

f_l = [5.5;
        -4.5;
        5.5;
        -4.5];

% For plotting
obs_x = [-f_l(2);
        f_l(1);
        f_l(1);
        -f_l(2);
        -f_l(2)];

obs_y = [-f_l(2);
        -f_l(2);
        f_l(1);
        f_l(1);
        -f_l(2)];
```

Define MPC Parameters

```
alpha_x = 0.1;
alpha_u = 0.000001;
num_states = 4;
num_controls = 2;
num_modes = 2; % const for now
num_faces = 4;

N = 11;
x_goal = [20; 0; 20; 0];

% Define the number of decision variables
num_t_N = 1;
num_t_x = N-1;
num_t_u = N-1;
num_x_p = N * num_states;
num_u_p = (N-1) * num_controls;
total_vars = num_t_N + num_t_x + num_t_u + num_x_p + num_u_p + num_modes+num_faces*num_modes;

num_constraints = 2*(num_x_p+num_u_p)+num_modes*num_x_p/2+2*N*num_faces+2;
```

Optimization

The cost function (objective)

$$J_p(\{x_p\}, \{u_p\}) = \|x_{\text{goal}} - x_p(k_p + N)\|_{\infty} + \sum_{j=k_p}^{k_p+N-1} \alpha_x \|x_{\text{goal}} - x_p(j)\|_{\infty} + \alpha_u \|u_p(j)\|_{\infty}$$

can be rewritten as a linear program

$$J_p = t_N + \sum_{j=k_p}^{k_p+N-1} \alpha_x t_x(j) + \alpha_u t_u(j)$$

with constraints

$$t_N \geq |x_{\text{goal},i} - x_p(k_p + N)| \quad \forall i$$

$$t_x(j) \geq \alpha_x |x_{\text{goal},i} - x_p(j)| \quad \forall i, \forall j \in [k_p, k_p + N - 1].$$

$$t_u(j) \geq \alpha_u |u_p(j)| \quad \forall i, \forall j \in [k_p, k_p + N - 1]$$

The MATLAB function `intlinprog` finds the minimum of a problem specified by

$$\min_x f^T x \text{ subject to } \begin{cases} x \text{ (intcon) are integers} \\ A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub. \end{cases}.$$

x is defined implicitly as

$$x = \begin{bmatrix} \mathbf{x}_p(k_p) : \mathbf{x}_p(k_p + N) \\ \mathbf{u}_p(k_p) : \mathbf{u}_p(k_p + N - 1) \\ t_x(k_p) : t_x(k_p + N - 1) \\ t_N \\ t_u(k_p) : t_u(k_p + N - 1) \\ d_i \\ b_a^i \end{bmatrix}$$

and f is defined as

$$f = \begin{bmatrix} 0 \cdot (\mathbf{x}_p(k_p) : \mathbf{x}_p(k_p + N)) \\ 0 \cdot (\mathbf{u}_p(k_p) : \mathbf{u}_p(k_p + N - 1)) \\ \alpha_x \cdot (t_x(k_p) : t_x(k_p + N - 1)) \\ t_N \\ \alpha_u \cdot (t_u(k_p) : t_u(k_p + N - 1)) \\ 0 \cdot d_i \\ 0 \cdot b_a^i \end{bmatrix}.$$

Optimization variable d_i is a binary variable to choose the low level control mode.

```
f = [zeros(num_x_p,1);
     zeros(num_u_p,1);
     alpha_x*ones(num_t_x,1);
     ones(num_t_N);
     alpha_u*ones(num_t_u,1);
     zeros(num_modes,1);
     zeros(num_modes*num_faces,1)];
```

The constraints

$$|x_{\text{goal},i} - x_p(k_p + N)_i| \leq t_N \quad \forall i$$

$$|x_{\text{goal},i} - x_p(j)_i| \leq t_x(j) \quad \forall i, \forall j \in [k_p, k_p + N - 1]$$

$$|u_p(j)_i| \leq t_u(j) \quad \forall i, \forall j \in [k_p, k_p + N - 1]$$

are implement using matrix A and vector b in the equation

$$A \cdot x \leq b.$$

The equations are rearrange to become compliant with the equation above:

$$\left[\begin{array}{l} -x_p(j)_i - t_x(j) \leq -x_{\text{goal},i} \quad \forall i, \forall j \in [k_p, k_p + N - 1] \\ -x_p(k_p + N)_i - t_N \leq -x_{\text{goal},i} \quad \forall i \\ u_p(j)_i - t_u(j) \leq 0 \quad \forall i, \forall j \in [k_p, k_p + N - 1] \\ x_p(j)_i - t_x(j) \leq x_{\text{goal},i} \quad \forall i, \forall j \in [k_p, k_p + N - 1] \\ x_p(k_p + N)_i - t_N \leq x_{\text{goal},i} \quad \forall i \\ -u_p(j)_i - t_u(j) \leq 0 \quad \forall i, \forall j \in [k_p, k_p + N - 1] \\ x_p(j)_i + M_{\text{big}} d_i \leq G + M_{\text{big}} \quad \forall m \in [x_p(j)_m = \text{velocity}], \forall j \in [k_p, k_p + N] \\ -E_a C_p x_p(k_p + j) - M_{\text{big}} b_a^i \leq -f_a^i \\ \sum_{a=1:q} b_a^i + d_i \leq q \end{array} \right]$$

```

%% Manipulating matrix A and b

A = zeros(num_constraints,total_vars);
index = 1; % helper indexer

% row 1 & 2
A(1:num_x_p,1:num_x_p) = -eye(num_x_p,num_x_p);
for i = 1:N
    A(index:index+num_states-1, num_x_p+num_u_p+i) = -ones(num_states,1);
    index = index + num_states;
end

% row 3
A(index:index+num_u_p-1,index:index+num_u_p-1) = eye(num_u_p,num_u_p);
for i = 1:N-1
    A(index:index+num_controls-1,num_x_p+num_u_p+N+i) = -ones(num_controls,1);
    index = index + num_controls;
end

% row 4 & 5
A(index:index+num_x_p-1,1:num_x_p) = eye(num_x_p,num_x_p);
for i = 1:N
    A(index:index+num_states-1, num_x_p+num_u_p+i) = -ones(num_states,1);
    index = index + num_states;
end

% row 6
A(index:index+num_u_p-1,num_x_p+1:num_x_p+num_u_p) = -eye(num_u_p,num_u_p);
for i = 1:N-1
    A(index:index+num_controls-1,num_x_p+num_u_p+N+i) = -ones(num_controls,1);
    index = index + num_controls;
end

d_start_index = num_x_p+num_u_p+num_t_x+num_t_N+num_t_u+1;
% row 7 & 8
M_big = 1000;
A(index:index+num_x_p/2-1,d_start_index) = M_big*ones(num_x_p/2,1);
for i = 2:2:num_x_p
    A(index,i) = 1;
    index = index + 1;
end
A(index:index+num_x_p/2-1,d_start_index+1) = M_big*ones(num_x_p/2,1);
for i = 2:2:num_x_p
    A(index,i) = 1;
    index = index + 1;
end

for i = 1:num_states:num_x_p
    A(index:index+num_faces-1,i:i+num_states-1) = -E_l*C_p;
    A(index:index+num_faces-1,d_start_index+num_controls:d_start_index+num_controls+num_faces-1) = -M_big*eye(num_faces,num_faces);
    index = index + num_faces;
end

for i = 1:num_states:num_x_p
    A(index:index+num_faces-1,i:i+num_states-1) = -E_l*C_p;
    A(index:index+num_faces-1,d_start_index+num_controls+num_faces:d_start_index+num_controls+2*num_faces-1) = -M_big*eye(num_faces,num_fa
    index = index + num_faces;
end

A(index:index+1,d_start_index:d_start_index+1) = eye(num_modes,num_modes);
A(index,d_start_index+num_modes:d_start_index+num_modes+num_faces-1) = ones(1,num_faces);
A(index+1,d_start_index+num_modes+num_faces:d_start_index+num_modes+2*num_faces-1) = ones(1,num_faces);

b = zeros(num_constraints,1);
index = 1;
for i = 1:N
    b(index:index+num_states-1) = -x_goal;
    b(index+num_x_p+num_u_p:index+num_x_p+num_u_p+num_states-1) = x_goal;
    index = index + num_states;
end
index = index+num_x_p+2*num_u_p;
G1 = 50;
G2 = 100;
b(index:index+num_x_p/2-1) = G1 + M_big;
index = index + num_x_p/2;
b(index:index+num_x_p/2-1) = G2 + M_big;
index = index + num_x_p/2;

for i = 1:2*N
    b(index:index+num_faces-1) = -f_l;
    index = index + num_faces;
end

```

```

end
b(index:index+1) = num_faces*ones(num_modes,1);

```

To implement the double integrator dynamics, we use the equality constraint

$$Aeq \cdot x = beq.$$

We manipulate Aeq and beq to have equations

$$\left[\begin{array}{c} x_p(k_p + j + 1) - A_p x_p(k_p + j) - B_p u_p(k_p + j) \quad \forall j \in [0, \dots, N - 1] \\ x_p = \text{initial condition} \\ \sum_i d_i = 1 \end{array} \right]$$

```

%% Manipulating Aeq and beq

Aeq = zeros(N*num_states+1, total_vars);

% Inserting x_p(k_p+j+1)
Aeq(1:(N-1)*num_states,num_states+1:N*num_states) = eye((N-1)*num_states,(N-1)*num_states);

% Inserting A_p in every equation
index = 1;
for i = 1:N-1
    Aeq(index:index+num_states-1,index:index+num_states-1) = -A_p;
    index = index + num_states;
end

% Initial condition
Aeq(index:index+num_states-1,1:num_states) = eye(num_states,num_states);
index = index + num_states;

%
Aeq(index,d_start_index:d_start_index+num_modes-1) = ones(1,num_modes);

% Inserting B_p in every equation
index_row = 1;
index_col = num_states*N+1;
for i = 1:N-1
    Aeq(index_row:index_row+num_states-1,index_col:index_col+num_controls-1) = -B_p;
    index_row = index_row + 4;
    index_col = index_col + 2;
end

beq = zeros(N*num_states+1,1);
beq(end) = 1;

```

Simulation

```

% Solve the linear program
sim_time = 100;
out = zeros(sim_time, num_states);
x_ref = zeros(M*sim_time, num_states);

for i = 1:sim_time
    sol = intlinprog(f, d_start_index:total_vars, A, b, Aeq, beq);
    out(i,:) = sol(5:8)';
    beq((N-1)*num_states+1:N*num_states) = sol(5:8);
    % calculate xref
    for j = 1:M
        x_ref((i-1)*M+j,:) = x_ref((i-1)*M+j,:) + (A_c^j*sol(1:4))';
        for m = 0:j-1
            x_ref((i-1)*M+j,:) = x_ref((i-1)*M+j,:) + (A_c^m*B_c*sol(num_x_p+1:num_x_p+num_controls))';
        end
    end
end
end

```

Running HiGHS 1.6.0: Copyright (c) 2023 HiGHS under MIT licence terms

Presolving model

282 rows, 89 cols, 624 nonzeros

262 rows, 85 cols, 582 nonzeros

Solving MIP model with:

262 rows

85 cols (1 binary, 8 integer, 0 implied int., 76 continuous)

582 nonzeros

Nodes		B&B Tree		Objective Bounds		Dynamic Constr	
Proc.	InQueue	Leaves	Expl.	BestBound	BestSol	Gap	Cuts InLp C
0	0	0	0.00%	-466.0005556	inf	inf	0 0

Plotting

```

plot(1:length(out), out(:,1));
hold on;
plot(1:length(out), out(:,3));
hold off;

plot(out(:,1), out(:,3));
hold on;
plot(obs_x,obs_y);
hold off;

```